



《AI大模型Ragent项目》——定时同步的调度引擎与故障恢复

来自： 拿个offer-开源&项目实战



马丁

2026年03月31日 22:45

上一篇讲了定时同步机制的基础设施：数据模型、五个核心组件、分布式锁、变更检测。这些是怎么设计的。
这一篇讲怎么跑起来的：调度引擎如何扫描任务、执行流程的三个阶段、任务生命周期管理、异常恢复机制、配置调优和常见问题。

调度引擎

理解了组件职责，再来看调度引擎的具体实现。

1. 调度引擎要解决的核心问题

调度引擎的核心职责是：周期性扫描到期任务，并提交到线程池异步执行。

这里有两个关键点：

- 周期性扫描**：不能让任务一直等着，也不能扫描太频繁浪费资源。默认配置是每 10 秒扫描一次，这是一个平衡点。
- 异步执行**：扫描和执行要分离。扫描线程只负责找到任务并提交到线程池，真正的刷新逻辑在线程池里异步执行。这样扫描线程不会被阻塞，可以继续扫描下一批任务。

2. 扫描到期任务

KnowledgeDocumentScheduleJob.scan() 方法每隔一段时间（默认 10 秒）执行一次。

先看查询条件，这是整个调度引擎的核心：

```
SELECT * FROM t_knowledge_document_schedule
WHERE enabled = 1
      AND (nextRunTime IS NULL OR nextRunTime <= now)
      AND (lockUntil IS NULL OR lockUntil < now)
ORDER BY nextRunTime ASC
LIMIT 20
```

这个查询的设计是找出到期且没人正在跑的任务。

逐个解读查询条件：

- enabled = 1**：任务处于启用状态。如果任务被禁用了（比如文档已删除、来源不再是 URL），就不应该再扫描到它。
- nextRunTime IS NULL OR nextRunTime <= now**：已经到执行时间了。nextRunTime IS NULL 表示首次执行，还没计算过下次执行时间。
- lockUntil IS NULL OR lockUntil < now**：当前没有其他实例持有锁。lockUntil IS NULL 表示锁不存在，lockUntil < now 表示锁已过期。
- ORDER BY nextRunTime ASC**：优先执行最早到期的任务。如果有多个任务同时到期，按照到期时间排序，先执行最早的。
- LIMIT 20**：每次最多扫描 20 个任务（可配置）。防止一次扫描太多任务，导致线程池被打满。

这个查询的精妙之处在于：它把到期判断和锁判断合并在一起，一次查询就能找出在该跑且没人正在跑的任务。

3. 获取锁与提交执行

扫描到任务后，不是直接执行，而是先尝试获取锁：

```
for (KnowledgeDocumentScheduleDO schedule : schedules) {
    // 尝试获取锁
    ScheduleLockLease lease = lockManager.tryAcquire(schedule.getId(), now);
    if (lease == null) {
        continue; // 抢锁失败，跳过
    }

    // 提交到线程池异步执行
    try {
        knowledgeChunkExecutor.execute(() -> scheduleRefreshProcessor.process(lease));
    }
}
```

```
    } catch (RejectedExecutionException e) {  
        // 线程池满了，释放锁  
        log.error("线程池已满，释放锁: scheduleId={}", schedule.getId(), e);  
        lockManager.release(lease);  
    }  
}
```

这段代码有两个关键设计：

抢锁失败直接跳过：如果 tryAcquire 返回 null，说明其他实例已经抢到这个任务了，当前实例直接跳过，不等待。这是租约锁的特点：不等待，抢不到就放弃。

线程池满了要释放锁：如果线程池已满，任务提交失败，必须立刻释放刚拿到的 lease。否则这个任务会白白卡住到锁过期（默认 15 分钟），期间其他实例也抢不到锁，任务就被耽误了。

这里体现了一个工程上的细节：异常处理不只是记录日志，还要做补偿动作（释放锁）。

4. 为什么要用线程池

你可能会问：为什么不直接在扫描线程里执行刷新逻辑？

因为刷新逻辑可能很耗时：

下载远程文件：可能几秒到几十秒

执行分块：可能几秒到几分钟

上传对象存储：可能几秒

如果在扫描线程里同步执行，扫描线程会被阻塞，无法继续扫描下一批任务。

用线程池异步执行的好处：

扫描线程不会被阻塞，可以继续扫描

多个任务可以并发执行，提高吞吐量

线程池可以限制并发数，防止资源耗尽

这是典型的生产者-消费者模式：扫描线程是生产者，线程池是消费者。

定时任务执行流程

前面讲了各个组件的职责，现在来看它们如何协作完成一次完整的定时同步。ScheduleRefreshProcessor.process() 方法是整个流程的编排者。

1. 流程概览

先建立整体认知。一次完整的定时同步可以分成三个阶段：

准备阶段（确认这次任务值不值得跑）：

1. 启动心跳
2. 校验任务合法性
3. 创建执行记录
4. 变更检测

执行阶段（真正开始刷新）：

5. 防并发冲突
6. 锁失效检测
7. 抢占文档运行权
8. 上传新文件
9. 锁失效检测
10. 执行分块
11. 验证分块结果
12. 应用文件元数据

收尾阶段（更新状态和清理资源）：

13. 标记成功
14. 异常处理
15. finally 文件清理

接下来逐步讲解每个阶段。

2. 准备阶段：确认任务值不值得跑

2.1 启动心跳

处理开始时先启动心跳，确保后面的刷新流程不会因为锁过期而中断。

为什么要第一步就启动心跳？因为后面的校验、变更检测、分块都可能很耗时，如果不启动心跳，锁可能会过期。

如果心跳启动失败，立刻释放锁并终止执行。不能让锁白白卡到 TTL 到期。

2.2 校验任务合法性

虽然任务在扫描阶段被找出来了，但真正执行时，环境可能已经变了。需要重新确认这些前提还成立：

- schedule 记录还存在吗？（可能被删除了）
- 文档还存在吗？（可能被删除了）
- 文档还启用吗？（可能被禁用了）
- 来源还是 URL 吗？（可能被改成本地文件了）
- 定时同步还开启吗？（可能被关闭了）
- cron 表达式还合法吗？（可能被改成无效的了）

如果这些前提不成立，这次任务就不该继续跑了。代码会调用 `disableIfOwned()` 自动禁用 schedule，而不是硬着头皮往下执行。

这是一个工程上的细节：不要假设环境不会变，要在执行前重新确认。

2.3 创建执行记录

每次执行一开始，都会先插一条 `RUNNING` 的 `exec` 记录。

为什么要先插记录？因为即使中途失败，后面也能追踪到这次执行到底停在了哪里。如果不插记录，失败后就没有任何痕迹，无法排查问题。

`exec` 记录包含：`scheduleId`、`docId`、`kbId`、`status` (`RUNNING`)、`startTime`。

2.4 变更检测

然后才去拉远程文件并做变更检测。这是准备阶段的最后一步，也是最关键的一步。

如果远程文件没变，这次就不做任何分块，直接记成 `SKIPPED`，更新 `exec` 记录和 `schedule` 主表，然后返回。

这里当前记录的是不管有没有文件更新，都保存运行记录。优点是可追溯，缺点是数据多了可能日志记录会比较多。有两个选型：

- 如果文件没有更新，程序里就不要再新增数据库记录。
- 再写个定时任务，删除x多长时间之间的执行记录。

两个策略都可以，不过如果文件不多，定时任务策略配置克制点，基本上不需要。

这里的关键不是定时到了就一定刷新，而是定时到了，先判断值不值得刷新。

大部分情况下（比如 90% 以上），文件都没变化，会在这一步直接返回。只有少数情况下文件有变化，才会进入执行阶段。

3. 执行阶段：真正开始刷新

如果变更检测发现文件有变化，就进入执行阶段。

3.1 防并发冲突（第一层）

远程文件确认有变化之后，并不会立刻上传和分块，而是先检查文档状态。

如果文档当前已经是 `RUNNING`，说明可能有手动分块正在进行，这次定时同步应该跳过。

为什么要检查？因为定时同步和手动分块可能同时触发。如果不检查，两个流程会同时操作同一份文档，导致冲突。

这是第一层防护：先看看文档是不是正忙着。

3.2 锁失效检测（第一次）

在抢占文档运行权之前，先检查 `lease` 是否还有效。

为什么要检测？因为前面的校验、变更检测可能花了一些时间，锁可能已经失效了。如果不检测，可能会抢占文档运行权后才发现锁失效了，白白占用了文档。

这是第一次锁失效检测。如果锁已经失效，立即中止执行，调用 `markLeaseLost()` 更新 `exec` 记录。

3.3 抢占文档运行权（第二层防并发）

通过第一层检查后，用 `CAS` 抢占文档的 `RUNNING` 状态。

`CAS` 操作的条件是 `ne(status, RUNNING)`，只有非 `RUNNING` 状态才能抢占成功。

为什么需要两层防护？因为第一层检查和第二层抢占之间有时间差，可能其他线程在这个间隙把文档占成 `RUNNING` 了。`CAS` 操作确保原子性。

抢占成功后，标记 `phase = DOC_OCCUPIED`。这个 `Phase` 标记很重要，后面会根据它决定清理策略。

3.4 上传新文件

拿到文档运行权之后，把新的远程内容上传到对象存储。

这里一个很重要的设计点是：**先上传，先分块，最后再切换文档主记录中的文件元数据。**

也就是说，上传成功不代表用户立刻看到的是新文件。只有后面的分块也成功了，系统才会把 `fileUrl` 等字段切到新文件上。

为什么要这样设计？因为如果上传后立刻切换，但分块失败了，用户看到的文档就是新文件但没有 `chunk`，查询时什么都查不到。

先上传、先分块、最后切换，确保用户看到的文档始终是有 `chunk` 可查的状态。

3.5 锁失效检测（第二次）

在执行分块之前，再次检查 `lease` 是否还有效。

为什么要再检测一次？因为上传文件可能花了一些时间，锁可能已经失效了。分块是最耗时的操作，必须确保锁仍然有效。

这是第二次锁失效检测。如果锁已经失效，标记 `leaseLost = true`，调用 `markLeaseLost()` 更新 `exec` 记录，然后返回。

3.6 执行分块

标记 `phase = CHUNK_STARTED`，然后执行分块。

这里没有单独再造一套定时同步版分块流程，而是直接复用前一篇文章里的 `documentService.chunkDocument()`。

设置 `UserContext` 为 `SYSTEM_USER`，表示这是系统自动触发的分块，不是用户手动操作。

分块流程会：

- 提取文本
- 分块
- 计算 Embedding
- 写入 `chunk` 表
- 写入向量数据库
- 更新文档状态为 `SUCCESS`

3.7 验证分块结果

分块结束后，重新查文档状态，确认它已经是 `SUCCESS`。

为什么要重新查？因为分块流程可能失败，文档状态可能是 `FAILED`。只有确认分块确实成功，才会继续往下走。

如果分块失败，调用 `markFailedIfOwned()` 更新 `schedule` 和 `exec` 记录，然后返回。

3.8 应用文件元数据

标记 `phase = CHUNK_COMPLETED`，然后切换文档主记录。

`applyRefreshedFileMetadata()` 把文档主记录里的 `docName / fileUrl / fileType / fileSize` 切换成新文件对应的值。

只有执行了这一步，新文件才真正生效。对查询侧和下载侧来说，这份文档此时才是新版本生效。

标记 `phase = FILE_SWITCHED` 表示文件已经正式切换，后面清理时应该删除旧文件。

4. 收尾阶段：更新状态和清理资源

4.1 标记成功

主流程收尾时，调用 `markSuccessIfOwned()` 写回调度结果。

这个方法会更新：

- `schedule` 主表：`lastRunTime`、`nextRunTime`、`lastSuccessTime`、`lastStatus`、`lastEtag`、`lastModified`、`lastContentHash`
- `exec` 记录：`status`、`endTime`、`message`、`fileName`、`fileSize`、`contentHash`、`etag`、`lastModified`

注意这个方法会检查 `lockOwner` 是否匹配当前 `lockToken`。如果不匹配（锁已失效），`schedule` 主表更新会失败，但仍会更新 `exec` 记录。

4.2 异常处理

`catch` 块处理所有异常，根据 `Phase` 分两种情况：

`phase != FILE_SWITCHED`（文件还没切换成功）：

- 如果已经抢占了文档运行权（`phase >= DOC_OCCUPIED`），恢复文档状态
- 调用 `markFailedIfOwned()` 更新 `schedule` 和 `exec` 记录

`phase == FILE_SWITCHED`（文件已经切换成功）：

- 分块已经成功了，只是写回调度状态时出错
- 不能把文档状态改成 `FAILED`，因为分块确实成功了
- 只调用 `markSuccessExecOnly()` 更新 `exec` 记录
- 记录错误日志，方便人工介入

4.3 finally 文件清理

`finally` 块负责清理资源和文件，这是整个流程最精细的部分。

关闭心跳：停止后台心跳线程。

恢复文档状态：如果锁已失效，且刚抢占完文档运行权但尚未进入分块（`phase == DOC_OCCUPIED`），恢复文档状态。

根据 Phase 精细化清理文件：

Phase	含义	清理策略	原因
FILE_SWITCHED	新文件已经正式生效	删除旧文件	新文件已经在用，旧文件可以删了
< CHUNK_COMPLETED	新文件还没真正用上	删除新文件	新文件上传了但没用上，删掉避免浪费存储
>= CHUNK_COMPLETED && < FILE_SWITCHED	分块成功但文档主记录还没切到新文件	保留新文件，打告警	chunk 和 vector 已经写入数据库了，删除新文件会导致数据不一致

这就是 Phase 存在的真正价值。它不是为了抽个枚举而抽象，是为了让补偿逻辑有明确依据。

释放锁：最后统一释放锁，确保锁一定会被释放。

到这里，一次完整的定时同步流程就讲完了。核心要点是：

准备阶段确认任务值不值得跑

执行阶段有两层防并发、两次锁失效检测

收尾阶段根据 Phase 精细化清理文件

定时任务生命周期

定时任务的生命周期和文档操作联动。

1. 创建

在 `startChunk` 的 `RocketMQ` 本地事务回调里，会调用 `scheduleService.upsertSchedule(documentD0)` 注册定时任务。

创建逻辑的核心是：

只处理 URL 来源的文档（非 URL 直接 return）

`enabled` 的计算：文档已启用 AND `cron` 非空 AND `scheduleEnabled=1` AND 间隔 `>= minIntervalSeconds`

按 `docId` 查 `schedule` 记录，有则 `update`，没有则 `insert`

`cron` 合法性校验：检查最小间隔是否满足要求

为什么要校验最小间隔？防止用户配置过于频繁的 `cron`（比如每秒一次），保护系统资源。

2. 更新

文档启用、禁用时，会调用 `scheduleService.syncScheduleIfExists(documentD0)` 更新已有记录。

这里的特点是 `allowCreate = false`，只更新已有记录，不额外新建。

3. 删除

文档删除时调用 `scheduleService.deleteByDocId(docId)`，先删 `exec` 记录，再删 `schedule` 记录，并放在事务里完成。

为什么要先删 `exec`？从业务逻辑上，`exec` 记录依赖于 `schedule` 记录，先删子表再删主表是更好的实践。

RUNNING 状态恢复

还有一个工程上很实用的补丁：恢复长时间卡住的 `RUNNING` 文档。

1. 为什么需要恢复机制

进程崩溃时，文档可能卡在 `RUNNING` 状态。虽然锁会自动过期（默认 15 分钟），其他实例可以接管任务，但文档状态不会自动恢复。

如果文档一直卡在 `RUNNING`，会出现两个问题：

用户看不到结果（既不是 `SUCCESS` 也不是 `FAILED`）

用户没法重新触发分块（因为文档状态是 `RUNNING`）

所以需要 一个兜底机制：定期扫描卡住的文档，自动恢复状态。

2. 恢复逻辑

KnowledgeDocumentScheduleJob 每分钟执行一次恢复任务，把卡在 RUNNING 状态超过阈值（默认 30 分钟）的文档重置为 FAILED。
恢复条件：

- 状态是 RUNNING
- updateTime 早于当前时间减去阈值
- 批量重置为 FAILED

这一步的目的不是精确判断任务一定已经死了，而是做一个工程上的安全兜底。

3. 阈值配置

默认配置 running-timeout-minutes = 30，最小值 10 分钟。
为什么是 30 分钟？

- 正常的分块流程通常在几分钟内完成
- 30 分钟足够覆盖大文件的分块时间
- 如果超过 30 分钟还在 RUNNING，大概率是异常了

恢复后，用户可以手动重新触发分块。

Cron 表达式工具

CronScheduleHelper 工具类负责 cron 表达式的解析和校验。

1. 计算下次执行时间

使用 Spring 的 CronExpression.parse() 解析表达式，调用 expression.next(fromTime) 计算下次执行时间。
如果解析失败或计算失败，返回 null，表示 cron 表达式无效。

2. 校验最小间隔

计算相邻两次执行的间隔，和 minSeconds 比较。
为什么需要最小间隔校验？

- 防止用户配置过于频繁的 cron（比如每秒一次）
- 保护系统资源，避免频繁下载和分块
- 默认最小间隔 60 秒

如果间隔小于最小值，创建 schedule 时会把 enabled 设为 0，禁用这个任务。

配置项

KnowledgeScheduleProperties 定义了定时同步机制的所有配置项，前缀 rag.knowledge.schedule：

配置项	默认值	说明
scan-delay-ms	10000	扫描间隔（毫秒），控制 scan() 方法的执行频率
lock-seconds	900	锁持有时间（秒），最小 60 秒，默认 15 分钟
batch-size	20	每次扫描批量大小，控制单次扫描最多处理多少个任务
min-interval-seconds	60	cron 最小间隔校验（秒），防止配置过于频繁的定时任务
running-timeout-minutes	30	RUNNING 状态超时阈值（分钟），最小 10 分钟，超过这个时间会被自动恢复

配置示例：

```
rag:
  knowledge:
    schedule:
```

```
scan-delay-ms: 10000      # 每 10 秒扫描一次
lock-seconds: 900         # 锁持有 15 分钟
batch-size: 20            # 每次最多处理 20 个任务
min-interval-seconds: 60  # 最小间隔 1 分钟
running-timeout-minutes: 30 # 超过 30 分钟自动恢复
```

FAQ

1. 为什么用数据库锁而不是 Redis 分布式锁

数据库锁的优势：

- 不新增基础设施：**调度状态本来就存在数据库里，再加两个字段就能把锁也放进去
- 锁和业务数据在一起：**查询 schedule 表就能看到锁状态，不需要跨系统查 Redis
- 异常排查更方便：**出问题时，运维可以直接查数据库看锁状态，不需要再去查 Redis

数据库锁的劣势：

- 吞吐不如 Redis：**数据库的并发性能不如 Redis
- 依赖数据库性能：**如果数据库压力大，锁操作也会变慢

对这个场景来说，定时任务的扫描频率不高（默认 10 秒一次），数据库完全扛得住。架构上更收敛，不需要为了锁单独维护 Redis 高可用。

2. 为什么要引入自动心跳续锁机制

相比手动续锁的优势：

- 业务代码无感知：**业务逻辑不需要记住在哪里续锁，只需在关键阶段检测 isLost() 即可
- 续锁时机自动计算：**心跳间隔根据锁持有时间自动计算 (lockSeconds / 3)，不需要手动调整
- 失败自动标记：**心跳失败时自动标记 lost，业务逻辑只需检测标记即可
- 降低心智负担：**不需要在每个耗时操作前手动续锁，减少出错可能

如果没有自动心跳，业务代码需要在下载文件前、上传文件前、执行分块前都手动调用 renew(lease)，容易遗漏。

3. 心跳失败后如何处理

心跳失败时，心跳线程会调用 heartbeat.markLost() 标记锁已失效。

业务逻辑在关键阶段调用 shouldAbortForLeaseLoss() 检测：

- 1. 检查心跳是否已标记 lost
- 2. 主动尝试续锁一次

如果发现锁失效，立即中止执行并调用 stateManager.markLeaseLost() 更新 exec 记录。

关键阶段包括：

- 抢占文档运行权之前
- 执行分块之前

这两个阶段都是高成本操作，必须确保锁仍然有效。

4. 实例宕机了怎么办

双重保障：

锁自动过期：

- 锁有过期时间（默认 15 分钟）
- 即使持有锁的实例宕机了，锁也会自动过期
- 其他实例可以在锁过期后接管任务

RUNNING 状态定时恢复：

- recoverStuckRunningDocuments 每分钟扫描一次
- 把卡在 RUNNING 状态超过阈值（默认 30 分钟）的文档重置为 FAILED

用户可以手动重新触发分块

所以即使实例宕机，最多 30 分钟后，任务和文档状态都能自动恢复。

5. 远程服务器不返回 ETag 和 Last-Modified 怎么办

退化到内容哈希校验：

- 第一级 HEAD 请求失败或 Header 不可靠时，进入第二级
- 直接下载文件并计算 SHA-256
- 和上次的 lastContentHash 比对

虽然多了一次下载，但相比重新分块和调用 Embedding API，成本还是低很多。而且大部分情况下，第一级 HEAD 请求能命中，不需要进入第二级。

6. 定时任务和手动触发分块是否会冲突

不会冲突，有两层防护：

第一层：检查文档状态：

- 在变更检测之后，先检查文档状态是否为 RUNNING
- 如果是，直接标记 SKIPPED，不继续执行

第二层：CAS 抢占文档运行权：

- 调用 documentStatusHelper.tryMarkRunning(docId)
- 条件是 ne(status, RUNNING)
- 只有非 RUNNING 状态才能抢占成功

即使第一层检查通过了，第二层 CAS 操作也能确保原子性，防止并发冲突。

7. 为什么需要 Phase 追踪

Phase 追踪的核心价值是精细化控制文件清理策略。

不同阶段失败，后果完全不一样：

- 还没开始分块就失败了：新文件可以直接删掉，保留旧文件
- 分块已经成功了，但文档主记录还没切过去：不能贸然删新文件，因为 chunk 和 vector 已经写入数据库了
- 文件已经切换成功了：应该删旧文件，而不是删新文件

Phase 枚举给 finally 块的清理逻辑提供了可靠坐标，没有 Phase 追踪，清理逻辑就无法准确判断应该删什么、留什么。

8. 锁失效时为什么只更新 exec 记录不更新 schedule 记录

锁失效说明其他实例可能已经抢到锁并更新了 schedule 记录。
如果当前实例强行更新 schedule 主表，可能会覆盖其他实例的状态，导致数据不一致。
所以 markSuccessIfOwned、markFailedIfOwned、markSkippedIfOwned 这些方法更新 schedule 主表时都会检查 lockOwner 是否匹配。如果不匹配，schedule 主表更新失败，但仍会更新 exec 记录。
exec 记录是本次执行的历史，不会和其他实例冲突。即使 schedule 主表写不进去，至少 exec 记录能留下这次执行的痕迹。

9. 为什么需要执行历史表

定时任务可能执行很多次，每次的结果不一样：

- 有时文件没变化，SKIPPED
- 有时文件有变化，SUCCESS
- 有时下载失败，FAILED

如果只在 schedule 主表保留“最近一次”的结果，就无法追溯历史：

- 这个任务过去一周执行了多少次？
- 成功率是多少？
- 哪些时间段经常失败？
- 失败原因是什么？

exec 表保留完整的执行历史，方便排查问题和统计分析。

10. 分块完成但文件元数据切换失败时为什么保留新文件

分块已经成功说明：

- 新文件的 chunk 已经写入 t_knowledge_chunk 表
- 新文件的 vector 已经写入向量数据库
- 文档状态已经是 SUCCESS

只是文档主记录的 fileUrl 等字段还没更新成新文件。
如果这时删除新文件，会导致：

- chunk 表里的记录指向一个不存在的文件
- vector 数据库里的向量对应的文件已经被删了
- 数据不一致

保留新文件，方便人工介入修复：运维可以手动把文档主记录的 fileUrl 改成新文件，或者重新触发一次分块。
删除新文件会导致数据不一致，而保留新文件只是多占用一点存储空间，风险更小。

小结

这一篇讲的是定时同步机制的执行层，核心要点：

- 调度引擎**：周期性扫描到期任务，用生产者-消费者模式异步执行，线程池满了要释放锁。
- 执行流程三阶段**：准备阶段确认值不值得跑（校验+变更检测），执行阶段有两层防并发+两次锁失效检测，收尾阶段根据 Phase 精细化清理文件。
- 生命周期管理**：创建（startChunk 时注册）、更新（文档启用/禁用时同步）、删除（文档删除时级联删除）。
- 异常恢复**：RUNNING 状态定时恢复（每分钟扫描，超过 30 分钟重置为 FAILED），配合锁自动过期（15 分钟）形成双重保障。
- 配置调优**：5 个核心配置项（扫描间隔、锁持有时间、批量大小、最小间隔、超时阈值），根据实际场景调整。

结合上一篇的基础设施（数据模型、组件职责、分布式锁、变更检测），你现在应该能完整理解这套定时同步机制是怎么设计的、怎么跑起来的。
回过头看，前面文章里的 startChunk 只是注册了定时任务，这两篇才把注册之后到底怎么跑起来补完整了。
到这里，知识库导入链路就比较完整了：

- 上传接口负责落对象存储和文档元数据
- startChunk 负责异步触发分块
- 定时同步机制负责让 URL 来源的知识持续保持最新

这样，知识库才不是一次性导入系统，而是一个能持续跟着源文档变化的动态系统。